darkmpc, Milestone 1 walkthrough (draft)

Prior work, secret sharing, protocol choices, threat model, and the reference matcher

Tave La Rocque

4 November 2026

Contents

Prior work: MPC dark pools **3**
- Why a dark pool needs this . . . 3
- What has been built . . . 3
- Their numbers . . . 3
- Where there is room to go further . . . 4
- Sources . . . 5

Secret sharing and three-party multiplication **6**
- The problem . . . 6
- Additive sharing . . . 6
- Replicated sharing . . . 6
- Restoring the replicated form, and why it needs fresh randomness . . . 7
- Worked example in $\mathbb{Z}_{256}$ . . . 8
- What this does and does not protect against . . . 8
- Sources . . . 9

Protocol decisions **10**
- 1. The ring: $\mathbb{Z}_{2^{64}}$ for data, $\mathbb{Z}_{2^{128}}$ for malicious checks . . . 10
- 2. Comparison: local conversion plus a log-depth carry circuit . . . 11
- 3. Malicious security with abort: optimistic multiply, then verify . . . 11
- What I am deliberately not deciding yet . . . 12
- Sources . . . 12

Threat model and leakage (draft) **14**
- Who is involved . . . 14
- What the adversary can do . . . 14
- Outputs, and how a trader reads them . . . 15
- What leaks by design . . . 15
- What must not leak . . . 15
- Leakage the functionality itself causes . . . 16
- Hiding symbol and side . . . 16
- What this document does not claim . . . 16

The reference matcher **17**
What it is for 17
The mechanism it defines 17
Why there is a pass cap 17
Two formulas that must agree 18
How the tests are built 18

Prior work: MPC dark pools

Every number below was checked against the paper's own text or tables (November 2026). Section and table references are to the versions linked at the end.

Why a dark pool needs this

In January 2016 Barclays and Credit Suisse agreed to pay a combined $154.3 million to settle investigations by the SEC and the New York Attorney General over their dark pools: Barclays paid $70 million and admitted misleading investors, and Credit Suisse paid $84.3 million ($60 million in penalties plus $24.3 million for other violations). The 2020 paper below lists further SEC settlements with dark pool operators through 2019. The common thread is that the operator sees every order. Remove the operator's view and that whole class of misconduct becomes impossible rather than merely prohibited.

What has been built

Cartlidge, Smart and Talibi Alaoui, "MPC Joins The Dark Side" (AsiaCCS 2019; ePrint 2018/1045). The first serious implementation. The authors build three mechanisms in the SCALE-MAMBA framework, all actively secure with abort, over a 128-bit prime field with 64-bit integers and 40-bit statistical security: a continuous double auction, a periodic auction, and a volume match. The two-party setting uses SPDZ (dishonest majority); the three-party setting uses Shamir sharing (the abstract calls it honest majority; Section 3 calls it "dishonest majority", which cannot be right for Shamir with three parties). They conclude the continuous double auction is not viable in MPC and that volume matching is. Their volume match is single-instrument, volume-only and first-come-first-served: total buys B and sells S, matched volume T = min(B, S), then walk each side in arrival order capping fills at what is left of T. There are no minimum fills. It costs 2(N+M)+1 comparisons and 9(max(N,M)+1) communication rounds (Sec. 4.3).

Cartlidge, Smart and Talibi Alaoui, Turquoise Plato Uncross (ePrint 2020/662). The same group scales the idea to the London Stock Exchange's Turquoise Plato Uncross venue: orders carry an instrument, side, size and minimum execution size (MES); the book is prioritised "by size and then time of order entry"; trades execute at the reference mid-price; and buy/sell pairs are matched sequentially, each pair checked against both MES constraints. That costs about 2NM comparisons per uncross (quadratic). Instruments are spread over L engines of R instruments each, so the system leaks which group of R instruments an order belongs to (they call this R-anonymity). Orders enter through distributed Paillier decryption. The design target is real: 4,500 instruments, an uncross every 5 seconds, about 2,000 orders per 5 seconds.

Their numbers

All benchmarks are LAN only: i7-7700K machines, 32 GB RAM, 10 Gbit switch, 0.47 ms ping.

Result	Two parties	Three parties	Source
Volume match, N = M = 500 (1,000 orders): online time	0.500 s	0.900 s	2018/1045, Table 8
same: offline (preprocessing) time	225.656 s	4.136 s	Table 8
same: online throughput (orders/s)	2,000	1,111	Table 8
same: rounds	4,509		Table 8
Volume match, N = M = 10: online / offline	0.010 / 4.623 s	0.018 / 0.100 s	Table 8
Headline in text	“800 orders per second”		Sec. 5
Plato uncross, R = 16, T(14,14)	about 2.5 s	about 3.6 s	2020/662, Sec. 5
Plato order insertion, R = 16	about 1.1 s	about 1.9 s	Sec. 5
Plato order input (Paillier), 2,000 orders on 64 cores	about 3.687 s		Sec. 5
Machines per organisation for 4,500 instruments, R = 16	282		Sec. 5

Two cautions. The 2018 paper’s text (“800 orders per second”) does not match its own Table 8 (1,000 to 2,200 online). And its “throughput” is online only: in the three-party case the 1,000-order batch also spends 4.136 s in preprocessing, so end to end it is closer to 200 orders per second.

Where there is room to go further

This part is my analysis, not a result from the papers.

Rounds, and therefore WAN. The 2018 volume match needs 9(max(N,M)+1) rounds because it walks the book sequentially. Neither paper benchmarks a WAN, and on a WAN, rounds dominate: 4,509 rounds at a 20 ms round trip is at least 90 seconds of waiting alone. The same first-come-first-served rule can be written as $fill_i = \min(size_i,\ \max(0,\ T - P_i))$, where P_i is the volume ahead of order i on its side. Prefix sums of secret-shared values are free local additions, so every fill can be computed in parallel in a constant number of rounds. The reference matcher already checks that this formula and the sequential walk agree.

Minimum fills without quadratic cost. Plato handles MES with about 2NM pairwise comparisons. With time priority, only the one marginal order on the longer side can violate its minimum in any pass, so elimination costs O(n) comparisons per pass and a small public number of passes. This is a different mechanism from Plato (time priority rather than size priority), so the honest comparison for minimum fills is a cost-and-leakage comparison, not a like-for-like benchmark.

What leaks. The 2018 system is single-instrument; the 2020 system leaks instrument groups by design. Hiding the symbol and side of every order, leaking only batch size and each trader’s own fill, is open.

Protocol choice and scale. Both papers run a general-purpose framework over a 128-bit prime field with a separate preprocessing phase. Replicated three-party sharing over a 64-bit ring makes semi-honest multiplication need no preprocessing and native machine arithmetic, and batches of 100 to 100,000 orders go well past their largest (1,000).

The fair comparison. The 2018 three-party volume match with every minimum fill set to 1 is exactly darkmpc's Stage 1 mechanism. Run with malicious security on a comparable LAN, it gives an apples-to-apples comparison against Table 8, as long as both online and end-to-end time are reported. darkmpc's semi-honest numbers must never be quoted against their actively secure ones.

Sources

- Cartlidge, Smart, Talibi Alaoui. MPC Joins The Dark Side. AsiaCCS 2019, pp. 148-159, doi:10.1145/3321705.3329809. ePrint: https://eprint.iacr.org/2018/1045 ; author manuscript read: https://core.ac.uk/download/226774922.pdf
- Cartlidge, Smart, Talibi Alaoui. Multi-Party Computation Mechanism for Anonymous Equity Block Trading: A Secure Implementation of Turquoise Plato Uncross. https://eprint.iacr.org/2020/662
- Settlement: CBS News / AP, "Banks reach $154.3 million settlement on dark pool fraud", 1 Feb 2016. https://www.cbsnews.com/news/banks-reach-154-3-million-settlement-on-dark-pool-fraud

Secret sharing and three-party multiplication

The problem

Three servers must compute the matching of a set of orders that none of them is allowed to see. The trick that makes this possible is older and simpler than most of cryptography: split each number into pieces that are individually meaningless, give each server some pieces, and find ways to compute on the pieces directly. Splitting a number this way is called secret sharing, and the pieces are shares.

All arithmetic here happens in the ring $\mathbb{Z}_{2^{64}}$, the integers modulo 2^64. That is exactly what a 64-bit unsigned integer does when you let it wrap around, so a Rust `u64` with `wrapping_add` and `wrapping_mul` is the ring, with no extra code. To keep the worked example readable it uses $\mathbb{Z}_{256}$ (8-bit wrapping arithmetic) instead. Nothing in the reasoning depends on the size.

Additive sharing

To share a secret x among three parties, pick x1 and x2 uniformly at random and set x3 = x − x1 − x2. Then x1 + x2 + x3 = x, so anyone holding all three shares recovers x by adding them.

Why are the shares meaningless on their own? Look at any two of them, say x1 and x2. They were chosen uniformly at random without looking at x, so they are uniform and tell you nothing. Now look at x2 and x3. Since x3 = x − x1 − x2 and x1 is a uniform value nobody has seen, x3 is uniform too, whatever x is: adding a uniform random value to anything in a ring gives a uniform result. This “one-time pad” argument is the whole security of secret sharing, and it is not computational. No amount of computing power helps someone who holds two of the three shares, because every value of x is exactly equally consistent with what they see.

Addition is free. If x is shared as (x1, x2, x3) and y as (y1, y2, y3), then each party adding its own two pieces produces (x1 + y1, x2 + y2, x3 + y3), which is a valid sharing of x + y. Multiplying by a public constant and adding a public constant are equally local. Nobody sends anything. This is why the matcher’s prefix sums cost nothing: a running total of secret sizes is just each server adding up its own shares.

Multiplication is not free. The product is xy = (x1 + x2 + x3)(y1 + y2 + y3), which expands into nine cross terms $x_i \cdot y_j$. In plain additive sharing, if each party holds just one share, no party holds both x1 and y2, so no one can compute that cross term without someone sending something. Every non-linear step in the matcher (comparisons, minimums, choosing between values) is built from multiplications, so the cost of multiplication is the cost of the whole system.

Replicated sharing

Replicated sharing is additive sharing where each party gets two of the three shares. Number the parties 1, 2, 3 and treat indices cyclically, so after 3 comes 1 again. Party i holds the pair (x_i, x_{i+1}): party 1 holds (x1, x2), party 2 holds (x2, x3), party 3 holds (x3, x1). Each share is held by exactly two parties.

Privacy is the same argument as before. Any single party sees exactly two of the three additive shares, which are uniform and independent of x. This is the honest-majority assumption made concrete: if two parties pooled their views they would have all three shares and learn x. The design is only secure while at most one of the three servers is corrupt.

Opening a value (deliberately revealing it, such as a trader's fill to that trader) needs one message per party: each party is missing exactly one share, and one of its neighbours has it.

The payoff is in multiplication. Party i holds x_i, x_{i+1}, y_i and y_{i+1}, so it can compute four of the nine cross terms itself. Give each party three of them:

$$z_i = x_i y_i + x_i y_{i+1} + x_{i+1} y_i$$

Party 1 covers x1y1, x1y2, x2y1. Party 2 covers x2y2, x2y3, x3y2. Party 3 covers x3y3, x3y1, x1y3. That is all nine terms, each exactly once, so z1 + z2 + z3 = xy. The product is now additively shared with one share per party, and nobody has communicated yet.

Restoring the replicated form, and why it needs fresh randomness

Two problems remain. First, each party holds only one share of the product, but the next multiplication needs the replicated form with two shares each. The fix is one message: party i sends z_i to party i − 1. Afterwards party i holds z_i (its own) and z_{i+1} (received from party i + 1), which is exactly the replicated pair. So a multiplication costs one round and one ring element sent by each party, which is what makes this protocol fast.

Second, and this is the subtle part, z_i as written leaks. The party that receives z_i is party i − 1, which already holds x_{i-1}, x_i, y_{i-1} and y_i. Substitute x_{i+1} = x − x_{i-1} − x_i and y_{i+1} = y − y_{i-1} − y_i into the formula for z_i and the terms rearrange into

$$z_i + x_i y_{i-1} + x_{i-1} y_i + x_i y_i = x_i\, y + y_i\, x.$$

Everything on the left is known to the receiver, so the receiver learns x_i·y + y_i·x: a linear equation in the two secrets with coefficients it knows. That is a real leak. For instance, if x_i happens to be odd and y_i even, the lowest bit of that quantity is the lowest bit of y, so the receiver learns whether y is odd.

The fix is to add a random sharing of zero before sending. The parties hold random values α1, α2, α3 with α1 + α2 + α3 = 0, each known only to its own party, and party i sends z_i + α_i instead. The sum is unchanged, so the result is still a sharing of xy, but to the receiver α_i is a uniformly random mask, so the message is uniformly random and carries no information. Forgetting this step is one of the bugs this project will plant on purpose, and the leakage tests must catch it.

The zero sharing costs no communication. Once, at setup, each pair of neighbouring parties agrees on a secret key: party i holds keys k_i and k_{i+1}, so each key is known to exactly two parties. For the c-th multiplication, party i computes α_i = F(k_i, c) −

F(k_{i+1}, c), where F is a pseudorandom function (in practice a standard cipher such as AES, from an audited library). Summing over all three parties, every F term appears once with a plus sign and once with a minus sign, so the α's sum to exactly zero. And the α_i that masks the message party i − 1 receives contains F(k_{i+1}, c), which party i − 1 does not hold the key for. This is the one place where the protocol's security is computational rather than information-theoretic: the masks are pseudorandom, not truly random.

Worked example in $\mathbb{Z}_{256}$

Let x = 20 and y = 7, so xy = 140. All arithmetic is mod 256.

Share x: pick x1 = 200 and x2 = 91, then x3 = 20 − 200 − 91 = −271 ≡ 241. Check: 200 + 91 + 241 = 532 = 2·256 + 20.

Share y: pick y1 = 130 and y2 = 45, then y3 = 7 − 175 = −168 ≡ 88. Check: 130 + 45 + 88 = 263 = 256 + 7.

The replicated holdings are: party 1 has x = (200, 91) and y = (130, 45); party 2 has x = (91, 241) and y = (45, 88); party 3 has x = (241, 200) and y = (88, 130).

The zero sharing: α1 = 17, α2 = 250, α3 = 245. Check: 17 + 250 + 245 = 512 ≡ 0.

Each party computes its masked product share:

z1 = 200·130 + 200·45 + 91·130 + 17 = 26000 + 9000 + 11830 + 17 = 46847 ≡ 255

z2 = 91·45 + 91·88 + 241·45 + 250 = 4095 + 8008 + 10845 + 250 = 23198 ≡ 158

z3 = 241·88 + 241·130 + 200·88 + 245 = 21208 + 31330 + 17600 + 245 = 70383 ≡ 239

Sum: 255 + 158 + 239 = 652 = 2·256 + 140. The shares reconstruct to 140, as they should.

Then party 1 sends 255 to party 3, party 2 sends 158 to party 1, and party 3 sends 239 to party 2. Afterwards party 1 holds (255, 158), party 2 holds (158, 239) and party 3 holds (239, 255): a replicated sharing of 140.

Now watch the leak without the mask. Unmasked, z1 = 26000 + 9000 + 11830 = 46830 ≡ 238, and party 3 receives it. Party 3 knows x3, x1, y3, y1, so it computes x1·y3 + x3·y1 + x1·y1 = 17600 + 31330 + 26000 = 74930 ≡ 178, and adds: 238 + 178 = 416 ≡ 160. The true value of x1·y + y1·x is 200·7 + 130·20 = 4000 ≡ 160. Party 3 has learned a fact about the secrets it should never have learned. With the mask, it receives 255 instead, which is uniformly random from its point of view.

(These numbers, and the leakage identity on 10,000 random sharings, were checked by script.)

What this does and does not protect against

Everything above assumes each server follows the protocol and only tries to learn from what it sees: the semi-honest model. A corrupt server could instead send a wrong z_i. Nothing in this protocol would notice, and the result would silently be

wrong. Detecting that (malicious security with abort) is the subject of Milestone 4, and the protocol choice for it is in the design decisions document.

Sources

- Araki, Furukawa, Lindell, Nof, Ohara. High-Throughput Semi-Honest Secure Three-Party Computation with an Honest Majority. CCS 2016. https://eprint.iacr.org/2016/768 . The protocol this chapter teaches. Their abstract: "each party sends only a single bit for every AND gate", over 1.3 million AES computations per second on three 20-core servers with 10 Gbps links. Their paper writes shares in an equivalent but differently indexed form (a pair of a share and a one-time-padded value); I have not yet read their ring construction in full text, so the exact notation here is mine.
- Lindell, Nof. A Framework for Constructing Fast MPC over Arithmetic Circuits with Malicious Adversaries and an Honest-Majority. CCS 2017, Sec. 6.2.1 (https://acmccs.github.io/papers/p259-lindellA.pdf): the same replicated structure, written "P1's share is (r1, r3), P2's share is (r2, r1) and P3's share is (r3, r2)". Only the indexing direction differs from this chapter.

Protocol decisions

This records the three core choices for darkmpc and the reasons, so each can be defended. Facts carry citations to sources that were opened and checked. Everything else is my judgement, labelled as such.

1. The ring: $\mathbb{Z}_{2^{64}}$ for data, $\mathbb{Z}_{2^{128}}$ for malicious checks

Decision. Secret values live in $\mathbb{Z}_{2^{64}}$, which is a Rust `u64` with wrapping arithmetic. In Milestone 4, the values used only for checking honesty are lifted to $\mathbb{Z}_{2^{128}}$ (a `u128`), giving 64 bits of slack above the data.

Why a ring and not a prime field. Three reasons, in order of importance.

The first is the workload. A dark pool matcher is comparison-heavy and multiplication-light. Each Stage 1 auction needs a few comparisons per order (a minimum and a clamp for each fill, a check against each minimum fill) and almost no other arithmetic. The cheapest known way to compare in three-party replicated sharing is local share conversion: each of the three additive summands of a ring element is held in the clear by two parties, so each summand can be turned into a Boolean sharing with no communication, and a binary adder circuit then recovers the bits of the sum. That trick depends on the sharing being plain additive sharing over a power-of-two ring. The MP-SPDZ documentation says local conversion "requires additive secret sharing over a ring without any MACs" and that "where available, local share conversion is likely the most efficient variant" (MP-SPDZ README, checked at commit f32f2f97). Over a prime field, addition of the summands wraps modulo p rather than modulo a power of two, so the bits of the sum are not the bits a binary adder produces, and you need a different, more expensive bit decomposition.

The second is speed and simplicity of the arithmetic itself: wrapping 64-bit integer arithmetic is what the CPU does natively, with no modular reduction.

The third is that it differs from prior work for a reason. Both MPC dark pool papers ran on a 128-bit prime field with a separate preprocessing phase (see prior-work.md). A ring-based design is a real point of comparison, not a cosmetic change.

What the ring costs. In $\mathbb{Z}_{2^{64}}$ most elements have no multiplicative inverse (every even number is a zero divisor). Two consequences follow. Division is awkward, which is one reason Stage 1 uses time priority rather than pro-rata allocation (pro rata needs a secret divisor). And the standard malicious-security checks over fields rely on a random challenge being invertible, which fails in a ring, so those checks have to be done in a bigger ring $\mathbb{Z}_{2^{k+s}}$ where the extra s bits absorb the problem. This is the SPDZ2k idea, and MP-SPDZ's malicious replicated ring protocols do exactly this: their check values have type `MaliciousRep3Share<Z2<K + S>>`, with S = 40 by default (Protocols/config.h). I choose s = 64 rather than 40 because k + s = 128 is a native `u128`; it costs a few extra bytes per checked value and gives more statistical slack. The precise soundness bound will be taken from the paper when Milestone 4 is implemented, not asserted now.

What makes comparison by sign sound. Comparing a and b by taking the top bit

of a − b in $\mathbb{Z}_{2^{64}}$ is only correct if a − b cannot wrap around. The reference matcher enforces sizes of at most 2^40 − 1 and batches of at most 2^17 orders, so every sum and difference the matcher computes stays strictly inside (−2^63, 2^63). The MPC matcher will enforce the same bounds (Milestone 4 must also check them for malicious clients, since a client can share anything).

2. Comparison: local conversion plus a log-depth carry circuit

Decision. To compute whether a < b, take d = a − b (free), convert d's three summands into Boolean replicated sharings locally, reduce three addends to two with one layer of full adders (a carry-save adder: one round of AND gates), then compute only the carry into the top bit with a parallel-prefix (log-depth) carry circuit. The sign bit is the answer. To use the result arithmetically (for example $\min(a, b) = b + [a < b] \cdot (a - b)$), convert the bit back into the ring with one more step (bit injection).

Why. It needs no preprocessing and no trusted dealer, its cost is a fixed number of AND gates and about log2(64) = 6 rounds of carry propagation plus a constant, and every piece is a small Boolean circuit that can be tested exhaustively at small bit widths. This is the approach ABY3 uses (its code computes the sum of two of the summands, then a two-input MSB extraction), and MP-SPDZ implements it for replicated rings (compiler option `-Z 3`), so MP-SPDZ can serve as an independent cross-check of results and of AND and round counts.

Alternatives considered. edaBits (Escudero et al., CRYPTO 2020) precompute random values shared both arithmetically and in binary, then compare by revealing a masked value. Their Table 8 reports, for a 63-bit comparison in honest-majority malicious $\mathbb{Z}_{2^k}$, 5.4 kbit with edaBits against 27.8 with daBits and 63.4 with plain arithmetic. That is attractive, but in the malicious setting it relies on large batches for the cut-and-choose to be efficient (MP-SPDZ notes bucket size 3 needs about a million edaBits per batch), and it adds a preprocessing phase to build and to prove correct. Rabbit (Makri, Rotaru, Vercauteren, Wagh, FC 2021) is a refinement of mask-and-reveal comparison with roughly 1.5× better throughput reported in most adversarial settings. Both are good upgrade paths once the simple version is measured; neither is the right first build.

Cost numbers. I am not quoting AND-gate or round counts for this circuit yet, because I could not verify them from a primary source. Milestone 2 will count them in our own implementation and cross-check against MP-SPDZ.

3. Malicious security with abort: optimistic multiply, then verify

Decision. Milestone 4 adds a verification layer on top of the unchanged semi-honest protocol. For ring multiplications, use the "multiply optimistically, then sacrifice-check" approach of Lindell and Nof (CCS 2017), with check values lifted to $\mathbb{Z}_{2^{128}}$. For the Boolean AND gates inside comparisons, use the triple-verification of Furukawa, Lindell, Nof and Weinstein (Eurocrypt 2017) as optimised by Araki et al. (IEEE S&P 2017). Inputs from clients get their own consistency check (see the threat model, still in draft). Any failed check makes every honest server abort before anything is opened.

Why this one. Three reasons. Its security argument is the simplest of the options: a cheating server that tampers with a product creates a triple that fails a randomised check except with small probability, and the check can be batched over all multiplications in an auction. It matches MP-SPDZ's `ps-rep-ring` protocol (which MP-SPDZ credits to Eerikson et al., ITC 2020, building on this line of work), giving a bit-for-bit cross-check. And it keeps local share conversion available, which the more communication-efficient SPDZ-wise approach does not, because SPDZ-wise attaches a MAC to every value.

The lineage, verified. Araki et al. (CCS 2016) gave the semi-honest protocol: one bit per AND gate per party. Furukawa et al. (Eurocrypt 2017) made it malicious with cut-and-choose verification of triples, about 10 bits per AND gate (as described in the S&P 2017 paper). Araki et al. (S&P 2017) reduced that to 7 bits per AND and reported about 1.15 billion AND gates per second on three 20-core Xeon servers over 10 Gbps. Lindell and Nof (CCS 2017) generalised to arithmetic circuits over fields, 4 field elements per multiplication per party in the replicated three-party case. Chida et al. (CRYPTO 2018) brought that to 2 field elements per multiplication for large fields with a single randomised check for the whole circuit. Abspoel, Dalskov, Escudero and Nof (ACNS 2021) carried that "SPDZ-wise" approach to $\mathbb{Z}_{2^k}$ at about twice the semi-honest cost. Boyle, Gilboa, Ishai and Nof (CCS 2019 and Asiacrypt 2020) replaced per-gate checks with sublinear distributed zero-knowledge proofs, getting to about one ring element per multiplication per party, matching semi-honest, with $\mathbb{Z}_{2^k}$ handled through an extension ring. Li et al. (CCS 2024) give sublinear checks for replicated sharing over $\mathbb{Z}_{2^\ell}$ without extension rings and report checking 10 million gates in 2.82 seconds.

What the current best practical approach is. On communication, the distributed zero-knowledge proofs win: verification becomes nearly free asymptotically, and the Li et al. CCS 2024 version is built for exactly our setting (replicated sharing over a power-of-two ring). That is the state of the art. It is also substantially harder to implement and to argue correct, so it is the stretch upgrade for Milestone 4 if the optimistic-multiply version is working with time to spare, and an honest line in the write-up if it is not.

What I am deliberately not deciding yet

Pseudorandom generation and channels will use audited libraries (a standard AES or ChaCha implementation for the PRF, rustls for TLS between servers and from clients), chosen in Milestone 2 after checking their APIs and audit status rather than from memory.

Sources

- MP-SPDZ, README and source at commit f32f2f97 (3 Nov 2026). https://github.com/data61/M
  SPDZ
- Araki et al. CCS 2016. https://eprint.iacr.org/2016/768
- Furukawa, Lindell, Nof, Weinstein. Eurocrypt 2017. https://eprint.iacr.org/2016/944
- Araki et al. Optimized Honest-Majority MPC for Malicious Adversaries. IEEE S&P 2017. https://www.ieee-security.org/TC/SP2017/papers/96.pdf

- Lindell, Nof. CCS 2017. https://acmccs.github.io/papers/p259-lindellA.pdf
- Chida et al. CRYPTO 2018; Journal of Cryptology 36:15 (2023). https://eprint.iacr.org/2018/57
- Abspoel, Dalskov, Escudero, Nof. ACNS 2021. https://eprint.iacr.org/2019/1298
- Boyle, Gilboa, Ishai, Nof. CCS 2019 (https://eprint.iacr.org/2019/1390) and Asiacrypt 2020 (https://eprint.iacr.org/2020/1451)
- Li, Escudero, Duan, Huang, Hong, Zhang, Song. CCS 2024. https://eprint.iacr.org/2024/700
- Eerikson et al. ITC 2020. https://eprint.iacr.org/2019/164
- Escudero, Ghosh, Keller, Rachuri, Scholl. edaBits. CRYPTO 2020. https://iacr.org/archive/cryp
- Makri, Rotaru, Vercauteren, Wagh. Rabbit. FC 2021. https://eprint.iacr.org/2021/119
- Mohassel, Rindal. ABY3. CCS 2018. https://eprint.iacr.org/2018/403 ; code https://github.com/ladnir/aby3

Threat model and leakage (draft)

This document says who darkmpc protects, from whom, and exactly what each party can learn. It is the contract the security argument and the leakage audit are checked against. Where a choice is still open it says so.

Who is involved

There are three kinds of party. **Traders** (clients) submit orders and receive their own fills. **Three servers**, run by three independent organisations, hold secret shares and run the matching; the intended deployment is something like the venue, a second market participant and a regulator or auditor, so that no two of them have a reason to collude. **The public** supplies two things everyone sees: the auction schedule (when each batch closes) and the reference midpoint from the lit market, at which every Stage 1 trade executes.

What the adversary can do

The adversary corrupts at most one server, chosen before the protocol starts (static corruption), and may also control any number of traders, including colluding with them. This is the honest-majority assumption: if two servers pool their shares, they see every order, and nothing in darkmpc prevents that. That limit belongs in the first paragraph of the README, not in a footnote.

In **Phase 1 (semi-honest)**, the corrupt server follows the protocol exactly and tries to learn from everything it sees: its shares, every message it receives, and the timing of both. The requirement is that its view can be produced by a simulator that knows only the public leakage listed below. In **Phase 2 (malicious with abort)**, the corrupt server may deviate arbitrarily: send wrong values, send nothing, or send different things to different parties. The requirement is that any deviation that would change an output is detected and every honest server aborts before any fill is revealed. darkmpc does not promise fairness or guaranteed output delivery: a corrupt server can always force an abort, so it can always deny service for an auction. That is an accepted limitation, not a bug.

Corrupt traders can submit anything at all, so every input check has to be done inside the MPC, on shares, without trusting the client. There are two separate attacks to stop.

The first is out-of-range values. A trader can share a size of 2^63 or a minimum fill larger than the size. Beyond producing nonsense, an out-of-range size breaks the soundness of comparison by sign (see decisions.md), which could corrupt other traders' fills. The servers must therefore check every bound the reference's `validate` checks (size in 1..2^40 − 1, minimum fill in 1..size, side a single bit, symbol in the public universe) on shares, and turn an invalid order into a dummy that matches nothing. The submitting trader may be told that its order was rejected. That reveals nothing about anyone else.

The second is inconsistent shares. In replicated sharing each additive share goes to

two servers. A malicious trader could give those two servers different values, so the three servers hold pieces that do not describe any single order, and different servers end up computing on different secrets. The fix is cheap: each pair of servers that should hold the same share compares a hash of what it received, and any mismatch rejects that order. In Phase 2 this check is mandatory. In Phase 1 traders are still untrusted, so it should be on from the start.

Outputs, and how a trader reads them

Each trader receives, for each of its orders, the filled quantity. Because every share is held by two servers, the trader receives each of the three shares of its fill twice. It checks that both copies agree and rejects the output if not. This stops a single corrupt server from feeding a trader a wrong fill at the last step.

Nothing else is published by default. In particular, per-symbol matched volume is not revealed: it would tell everyone where interest is resting, which is exactly what a dark pool exists to hide. A real venue would later report executed trades to a public tape under regulation. That is outside this prototype and is noted as such.

What leaks by design

Leaks	To whom	Why it is acceptable, or how it is limited
Each trader's own fills	That trader	The purpose of the venue
Number of orders in a batch	Servers	Can be hidden by padding every batch to a public size with dummy orders. Recommended; cost is linear in the padding
Which traders submitted, and when	Servers (network metadata)	Inherent to clients connecting. Batching per auction limits timing detail to "submitted before the cutoff"
Message sizes	Servers, network observers	Every order has the same fixed-size encoding, so sizes reveal only the order count
That an auction aborted	Everyone	An abort in Phase 2 means a server cheated or failed. It is visible by design
Rejection of an invalid order	Its submitter	Concerns only that trader's own input

What must not leak

To any single server, possibly together with corrupt traders: the symbol, side, size, minimum fill or even existence (beyond the public count) of any honest trader's order; whether any order was eliminated for its minimum fill; whether a symbol hit the pass-cap fail-safe; per-symbol totals; and any unmatched interest. The number of elimination passes is not a leak, because it is always the public cap k.

Leakage the functionality itself causes

Some leakage survives even a perfect implementation, because the ideal functionality hands each trader its fills, and fills carry information about others. A trader who submits a one-share buy in a symbol learns from a non-zero fill that sell interest exists there. Repeated across symbols and auctions, this is "pinging", a known way of gaming real dark pools, and MPC does nothing about it. The defences are market rules, not cryptography: minimum order sizes, fees, limits on how many orders each trader can send, and minimum fills on large resting orders (which make a one-share probe fail to fill). The write-up must say this plainly. darkmpc hides orders from the operator; it does not make a dark pool immune to its own participants.

Hiding symbol and side

Symbols and sides are secret, as decided on 4 November 2026. That changes the computation. With public symbols, the servers could split the batch into one small auction per symbol. With secret symbols they cannot, so grouping orders by symbol has to happen inside the MPC. There are two families of approaches, and the choice between them is the largest cost decision in Stage 1.

The first fixes a public universe of U symbols and has each trader share its symbol as a one-hot vector of U bits, so per-symbol totals become sums over all orders weighted by those bits. It is simple and has a fixed number of rounds, but its work grows with $n \times U$, which is prohibitive for thousands of symbols and large batches.

The second sorts the batch by symbol obliviously, computes per-symbol totals with segmented prefix sums over the sorted order, matches, and routes each fill back to its original position. Done naively with a sorting network, the work grows with $n \cdot \log^2 n$ comparisons. Shuffle-then-sort techniques first randomly permute the batch so that comparison results can be opened safely, bringing it down towards $n \cdot \log n$. The sort must be stable so that arrival order, and therefore time priority, survives within a symbol.

I am not quoting costs for either approach until they are checked against primary sources. The recommendation will come with the Milestone 2 design, alongside the cost of hiding side, which turns "volume ahead of order i on its side" into a sum weighted by secret same-side bits (one multiplication per pair of orders in naive form, linear with the sorted segmented-scan form).

What this document does not claim

It does not claim security against two colluding servers, against an adversary that corrupts servers adaptively during an auction, against denial of service, against manipulation of the lit-market midpoint that Stage 1 trades at, or against traffic analysis beyond message counts and sizes. The code is a research prototype and has not been audited. The empirical leakage tests planned for Milestone 5 are a check that catches implementation bugs, such as the missing re-randomisation bug that will be planted on purpose. They are not evidence of indistinguishability. The security argument is the simulation proof on paper.

The reference matcher

What it is for

Before any cryptography exists, darkmpc needs a definition of the right answer. The reference matcher is that definition: a short, plain Rust function that takes a batch of orders and returns each order's fill, with no secret sharing and no attempt at speed. Every later milestone is judged against it. In Milestone 3 the MPC matcher must return exactly the reference's fills on millions of random batches, and if the two ever disagree the MPC matcher is wrong by definition. That only works if the reference is simple enough to trust by reading it, so it is written the obvious way and kept in its own crate with no dependencies.

The mechanism it defines

An auction runs per symbol. Total the eligible buy sizes into B and the sell sizes into S; the matched volume is T = min(B, S), traded at the public midpoint, so no prices are involved. Each side then shares out T by time priority: the earliest order takes as much as it wants up to what is left, then the next, and so on. On the side with less volume everyone fills completely. On the side with more, a run of orders fills completely, then one order (the marginal order) gets a partial fill, then the rest get nothing.

Minimum fills complicate this. An order may say "fill me at least m shares or not at all". If the marginal order's partial fill falls below its minimum, it cannot trade, so it is removed and the allocation is redone. That can push the imbalance to the other side, whose marginal order may now be below its own minimum, so the process can bounce back and forth. Each round is a pass, and the process stops at the first pass with no violation.

Two facts make this cheap and well defined. Only the marginal order can ever violate its minimum, because full fills never violate (a minimum is at most the order's size) and everyone else gets zero, so each pass removes at most one order. And since each pass removes an order or stops, the process ends within n + 1 passes for n orders. The tests check both facts directly on random books rather than taking them on trust.

Why there is a pass cap

An MPC evaluation cannot run "until stable", because the number of passes it ran would itself leak information about the orders. It has to run a fixed, public number of passes k. So the reference implements the capped rule too: run at most k passes, and if pass k still has a violator, that symbol matches nothing this auction. The fail-safe means a minimum is never broken, whatever k is. The tests prove that a cap of k gives exactly the uncapped answer on every symbol that settles within k passes and all zeros on every symbol that does not. Choosing k is a trade-off between cost (each pass is a few comparisons per order) and how often the fail-safe triggers, and it will be measured on realistic order-size distributions before it is fixed.

Two formulas that must agree

Prior work allocates by walking down each side with a running remainder, which in MPC means one step after another: their volume match needs 9(max(N,M)+1) communication rounds. The same rule can be written as a closed formula. If P_i is the total size of the orders ahead of order i on its side, then order i's fill is $\min(size_i, \max(0, T - P_i))$. Prefix sums of secret-shared values are free local additions, so every fill can be computed at once, in a constant number of rounds whatever the batch size. The reference contains both forms, `allocate_walk` and `allocate_prefix`, and a property test requires them to be identical on random books. The matcher itself uses the prefix form, because that is what the MPC version will compute.

How the tests are built

There are 13 hand-worked examples, each with its expected answer worked out in a comment rather than copied from the code. The first is the worked example printed in "MPC Joins The Dark Side" (sells 3, 4, 5 against buys 5, 2, 11, 1), so the mechanism is pinned to the published one. The others cover an empty batch, an empty side, a perfectly balanced book, ties, a fill exactly at the minimum, removal of a marginal order, an all-or-none order that cannot trade, elimination bouncing between sides for three passes, the pass-cap fail-safe, two interleaved symbols, maximum sizes in a batch of 79,999 orders without overflow, and every kind of invalid input.

There are nine property tests, each run on 4,096 random batches by default (the run for this draft used 100,000): the walk and prefix formulas agree; bought volume equals sold volume on every symbol; every fill is zero or between the minimum and the size; time priority holds; at most one order per symbol is partially filled; without binding minimums, matched volume is exactly min(B, S); each pass has at most one violator and the uncapped rule never needs the fail-safe; capped runs equal uncapped runs or fail safe exactly when they should; and each symbol's fills ignore every other symbol's orders. The random batches mix tiny sizes (to force ties and binding minimums) with sizes at the maximum.

Finally, three bugs are planted in the code behind feature flags: a minimum check that is off by one at the boundary, an elimination step that is skipped, and a prefix sum that wrongly counts an order as ahead of itself. `scripts/check_planted.sh` builds each one and passes only if the test suite fails for every one of them, which shows the tests can actually fail.